

Patient Observation Tracker — Design Document

CSCI-P532 Object Oriented Software Development · Spring 2026 · Week 1

LAYERED COMPONENT DIAGRAM

CLIENT LAYER — @RESTCONTROLLER

PatientController · ObservationController
CatalogueController · LogController

MANAGER LAYER — @SERVICE (MANAGER)

PatientManager · ObservationManager
CatalogueManager · LogManager

ENGINE LAYER — @SERVICE (ENGINE)

DiagnosisEngine (uses DiagnosisStrategy)

RESOURCEACCESS LAYER — @REPOSITORY

PatientRepository · ObservationRepository
PhenomenonTypeRepository · PhenomenonRepository
ProtocolRepository · AssociativeFunctionRepository
CommandLogRepository · AuditLogRepository

RESOURCE (DOMAIN ENTITIES)

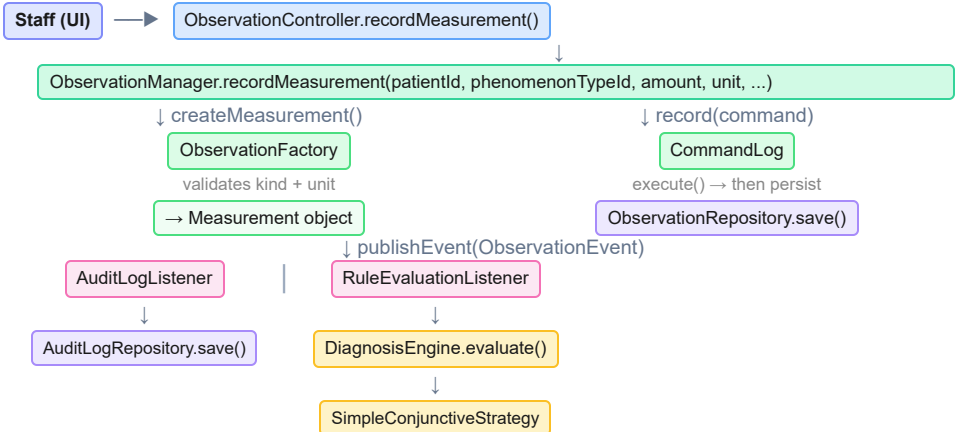
Patient · PhenomenonType · Phenomenon · Protocol
Observation (abstract) · Measurement · CategoryObservation
AssociativeFunction · CommandLogEntry · AuditLogEntry

UTILITY (CROSS-CUTTING)

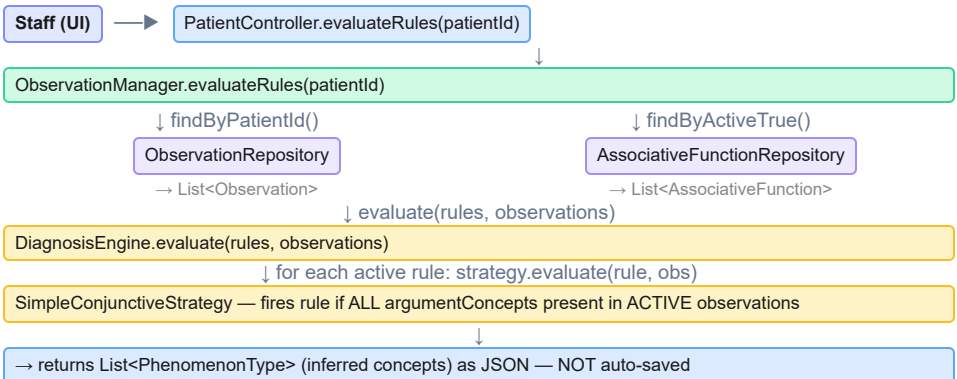
ObservationFactory · CommandLog · BaseCommand
SimpleConjunctiveStrategy · ObservationEvent
AuditLogListener · RuleEvaluationListener · DataInitializer

CALL CHAIN / SEQUENCE DIAGRAMS

Use Case 1 — Record Measurement (POST /api/observations/measurement)



Use Case 2 — Evaluate Diagnostic Rules (POST /api/patients/{id}/evaluate)



DESIGN PATTERNS — CLASSES & JUSTIFICATIONS

#	Pattern	Classes Involved	One-Sentence Justification
1	Factory	ObservationFactory, Measurement, CategoryObservation, PhenomenonType, Phenomenon	All Observation subtype objects are constructed exclusively through ObservationFactory, which enforces kind/unit invariants before any object is created, preventing invalid observations from ever entering the system.
2	Strategy	DiagnosisStrategy (interface), SimpleConjunctiveStrategy, DiagnosisEngine	The diagnosis algorithm is injected into DiagnosisEngine as a DiagnosisStrategy interface, allowing Week 2's WeightedScoringStrategy to be swapped in without modifying the engine at all.
3	Observer	ObservationEvent, ApplicationEventPublisher, AuditLogListener, RuleEvaluationListener, ObservationManager	ObservationManager publishes an ObservationEvent via Spring's event bus whenever an observation is created or rejected, fully decoupling audit logging and rule re-evaluation from the manager's core orchestration logic.
4	Command	Command (interface), BaseCommand, CommandLog, CommandLogEntry	Every state-changing action (create patient, record observation, reject) is wrapped in a BaseCommand with an execute() method and a JSON payload, which CommandLog persists after execution to support a full audit trail and future undo.
5	Decorator	ObservationProcessorDecorator (abstract base, Week 2 hook), DiagnosisEngine pipeline	An abstract ObservationProcessorDecorator with a delegate field is defined as the Week 2 extension point, so new processing behaviors (e.g. PropagationDecorator) can be added as wrappers without touching existing processor code.